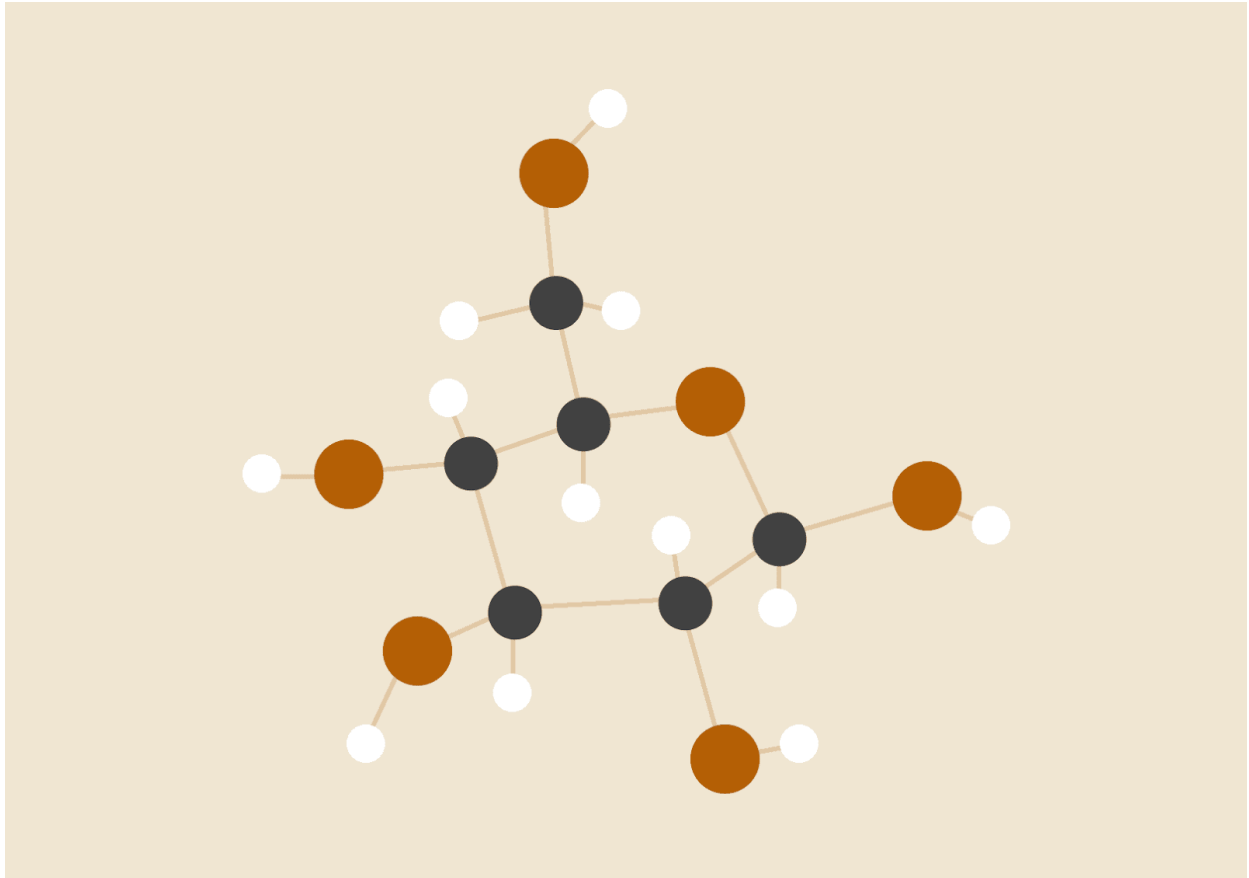


GRAPH DATABASE



Team - The Traveling Salesmen

Chiluveru Pranav Vardhan (22CS10019)

Parimi Nishnath (22CS10050)

Mupparaju Venkata Sai Varun (22CS10044)

Gavinikadi Aravind (22CS10024)

Amballa Saketh (22CS10007)

Github - <https://github.com/nishnath-209/GPS>

ABSTRACT

Graph databases have emerged as a powerful tool for managing and analyzing highly connected data structures. This project aims to investigate graph exploration and processing through various graph database and processing systems, including **Neo4j**, **Apache TinkerPop**, **Giraph**, **Pregel (GoldenOrb)**, and **Stanford GPS**. Our primary objective is to evaluate these systems by loading a complex, large-scale graph dataset from the **Stanford SNAP repository** and creating an intuitive interface to support the execution of fundamental graph queries.

Beyond basic queries, we intend to implement advanced graph algorithms such as **PageRank** and **community detection** to illustrate the analytical capabilities of these platforms. Performance profiling will be conducted to assess system scalability, efficiency, and responsiveness under varying loads and query complexities. Through these experiments, we hope to gain meaningful insights into the strengths and trade-offs of different graph processing paradigms.

This project will primarily utilize Neo4j, a widely adopted graph database platform, to achieve our goals of **data ingestion**, **query processing**, **algorithmic analysis**, and **performance benchmarking**. Neo4j provides the following advantages that align with our project goals:

- Scalable storage and processing of complex graph structures.
- **Cypher query language** for expressive and efficient querying.
- Flexibility to define and execute custom graph algorithms.
- Integrated monitoring and profiling tools for performance tuning.

By leveraging the features of Neo4j and optionally validating results with Stanford GPS, we aim to provide a comprehensive evaluation of graph database technologies for large-scale graph data management and analysis.

OBJECTIVES

Understanding Graph Databases

We aim to learn how graph databases like Neo4j represent and manage interconnected data using nodes and relationships. This approach simplifies complex data models and makes querying more intuitive compared to traditional databases.

Loading Graph Data & Running Graph Queries

We will create graphs with numeric nodes and real-life datasets like a movie database. These graphs will include interconnected entities like movies, actors, directors, and reviewers, and we'll load large-scale datasets to test Neo4j's performance.

The goal is to build an interface for executing basic graph queries. We'll allow users to search for movies by a director or find actors linked to certain genres, demonstrating the power of graph-based queries.

Applying Graph Algorithms

We'll implement algorithms like PageRank to rank nodes based on their connections. Additionally, we plan to explore community detection to identify tightly connected groups in the graph, enhancing our understanding of graph theory applications.

Analyzing Performance

We'll evaluate Neo4j's performance by monitoring metrics like query time and memory usage as the graph grows. This helps us understand its efficiency and scalability for large datasets.

Real-World Application: Movie Database

We will model a movie database where movies, actors, and directors are interconnected. This real-world graph will allow users to explore relationships, such as finding all movies by a director or discovering top-rated films based on reviewer ratings.

METHODOLOGY

(i) System Architecture

The system follows a client-server architecture with the following components:

Backend Server: A FastAPI-based Python application that serves as the middleware between the frontend and the Neo4j graph database. It handles data processing, query execution, and algorithm implementation.

Graph Database: Neo4j database instance that stores and manages graph data in two main schemas:

A general-purpose node-relationship graph. & a movie database with actors, directors, and relationships between them

Frontend: A React-based user interface that provides interactive controls for querying and visualizations of the graph data.

(ii) Data Model

The system supports two primary data models:

General Graph Model:

Nodes with numeric identifiers and labels

Directed relationships of type "**CONNECTS**" between nodes

Movie Database Model:

Movie nodes with properties like title, release year, and tagline

Person nodes with properties like name and birth year

Various relationship types including **ACTED_IN**, **DIRECTED**, and **PRODUCED**

(iii) Implementation Details

Backend Implementation

The backend was implemented using **FastAPI**, a modern Python web framework that enables rapid development of APIs with automatic documentation. Key features of the backend include -

Query Processing: Functions to parse and execute specialized graph queries.

Profile Analysis: Performance tracking of query execution through Neo4j's profiling capabilities.

Algorithm Implementation: Graph algorithms implemented using Cypher queries, including -

- a) Shortest path algorithms
- b) Community detection
- c) Centrality measures (PageRank, Degree Centrality)
- d) Triangle detection and clustering coefficient calculation
- e) Query Optimization

API Design:

The backend exposes the following key endpoints:

/get_data: Fetches the basic graph structure or movie database

/process_input: Processes analytical queries with parameters

/: Root endpoint that confirms API availability

(iv) Analytical Capabilities

The system implements a wide range of graph analytics capabilities:

a) Path Analysis:

- (i) Finding shortest paths between nodes
- (ii) Discovering all shortest paths
- (iii) Finding paths of specific length k
- (iv) Centrality Measures
- (v) Degree centrality

b) PageRank implementation

c) Community Structure

d) Triangle counting

e) Clustering coefficient calculation

f) Community detection

g) Domain-Specific Analysis (Movie Database)

h) Actor-movie relationship analysis

i) Finding co-actors and collaborations

j) Identifying central actors and influential directors

QUERIES

1) Basic graph structure queries

Query Type	Description	Cypher Query
Fetch data	Fetches all nodes and relationships	<code>MATCH (n:node) RETURN n and MATCH (a:node)-[r]->(b:node) RETURN a, r, b</code>
Fetch Movie Database	Fetches movie data with related people	<code>MATCH (m:Movie)-[r]-(p:Person) RETURN m.title AS movie, m.released AS released, m.tagline AS tagline, collect({name: p.name, born: p.born, relationship: type(r), roles: r.roles}) AS people</code>
Count total Nodes	Counts all nodes in the graph	<code>MATCH (n:node) RETURN n AS node, count(n) AS nodeCount</code>

2) Path analysis queries

Query Type	Description	Cypher Query
Find Shortest distance	Finds the shortest path between the nodes	<code>MATCH path = shortestPath((n1)-[:CONNECTS*..150]->(n2)) WHERE n1.num = {startNode} AND n2.num = {endNode} RETURN path, length(path) AS distance, nodes(path) AS pathNodes, relationships(path) AS pathEdges</code>
Find all shortest paths	Finds all the shortest paths between two nodes	<code>MATCH paths = allShortestPaths((n1)-[:CONNECTS*..150]->(n2)) WHERE n1.num = {startNode} AND n2.num = {endNode} RETURN paths, length(paths) AS distance, nodes(paths) AS pathNodes, relationships(paths) AS pathEdges</code>

Find k-length paths	Finds paths of exact length k between the nodes	MATCH path=(n1)-[:CONNECTS*{k}]->(n2) WHERE n1.num = {startNode} AND n2.num = {endNode} RETURN nodes(path) AS pathNodes, relationships(path) AS pathEdges
---------------------	---	---

3) Connectivity queries

Query Type	Description	Cypher Query
Find connected nodes from	Finds nodes connected from a specific node	MATCH (n {num: {inputValue}})-[:CONNECTS]->(connected) RETURN connected
Find connected nodes to	Finds nodes connected to a specific node	MATCH (connected)-[:CONNECTS]->(n {num: {inputValue}}) RETURN connected
Find common neighbours	Finds common neighbours between two nodes	MATCH (n1 {num: {node1}})--(common)--(n2 {num: {node2}}) RETURN common

4) Community and structure analysis

Query Type	Description	Cypher Query
Triangle count	Counts all the triangles in the graph	MATCH (a)-[:CONNECTS]->(b)-[:CONNECTS]->(c)-[:CONNECTS]->(a) RETURN count(DISTINCT [a, b, c]) AS triangle_count
Node triangles	Finds Triangles containing a specific node	MATCH (a)-[:CONNECTS]->(b)-[:CONNECTS]->(c)-[:CONNECTS]->(a) WHERE a.num = {node_id} OR b.num = {node_id} OR c.num = {node_id} RETURN DISTINCT a.num AS a, b.num AS b, c.num AS c

Clustering coefficient	Calculates clustering coefficient of a node	Multiple queries (see calculation function in code)
Community detection	Detects communities in the graph	Multiple iterative queries to assign community IDs
Find Isolated Nodes	Finds nodes with no connections	<code>MATCH (n) WHERE NOT (n)--() RETURN n.name AS name, labels(n) AS labels</code>

5) Centrality measures

Query Type	Description	Cypher Query
PageRank	Calculates PageRank for nodes	Multiple iterative queries(custom Implementation)
Degree Centrality	Calculates degree centrality	<code>MATCH (n)-[r:CONNECTS]-() WITH n, count(r) AS degree RETURN n.num AS node, degree AS centrality ORDER BY centrality DESC LIMIT 10</code>
Find most Central Actor	Finds actor with highest centrality	<code>MATCH (p:Person)-[:ACTED_IN]-() RETURN p.name AS actor, count(*) AS score ORDER BY score DESC LIMIT 1</code>

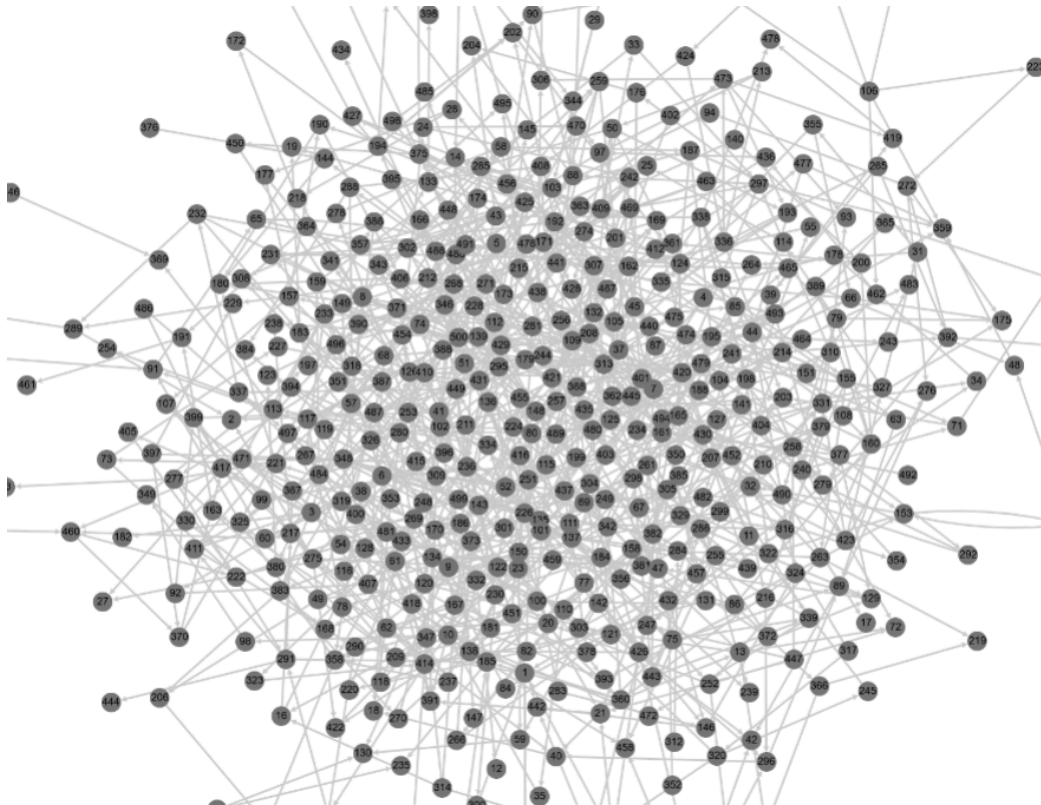
6) Movie database queries

Query Type	Description	Cypher Query
Fetch all movies	Gets all movies	<code>MATCH (m:Movie) RETURN m.title AS title, m.released AS released, m.tagline AS tagline</code>

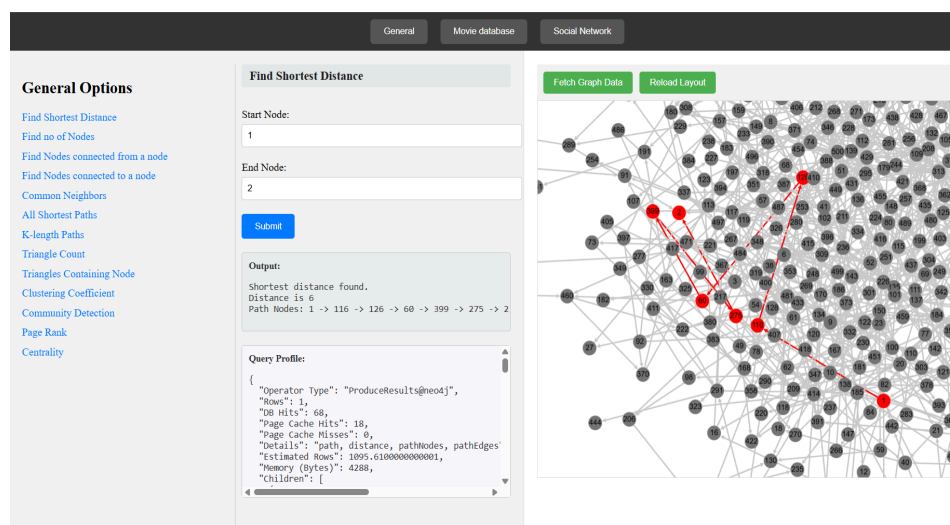
Fetch movie details	Fetches details of a specific movie	MATCH (m:Movie {title: '{movie_title}'})<-[r]-(p:Person) RETURN m.title AS title, m.released AS released, m.tagline AS tagline, collect({name: p.name, born: p.born, relationship: type(r), roles: r.roles}) AS people
Find Co-actors	Finds co-actors of a given actor	MATCH (a:Person {name: '{actor_name}'})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(co:Person) RETURN co.name AS coActor, collect(m.title) AS movies
Find Actor-Director movies	Finds movies with specific actor and director	MATCH (a:Person {name: '{actor_name}'})-[:ACTED_IN]->(m:Movie)<-[:DIRECTED]-(d:Person {name: '{director_name}'}) RETURN m.title AS movie
Find Directors for Actor	Finds directors who worked with actor	MATCH (a:Person {name: '{actor_name}'})-[:ACTED_IN]->(m:Movie)<-[:DIRECTED]-(d:Person) RETURN d.name AS director, collect(m.title) AS movies
Find frequent collaborators	Finds Actor-director pairs with multiple collaborators	MATCH (a:Person)-[:ACTED_IN]->(m:Movie)<-[:DIRECTED]-(d:Person) RETURN a.name AS actor, d.name AS director, count(m) AS collaborations ORDER BY collaborations DESC
Find Movies by Director	Finds movies directed by specific person	MATCH (d:Person {name: '{director_name}'})-[:DIRECTED]->(m:Movie) RETURN m.title AS movie, m.released AS released, m.tagline AS tagline
Find Actor Path	Finds shortest path between actors	MATCH path = shortestPath((a:Person {name: '{actor1}'})-[*]-(b:Person {name: '{actor2}'})) RETURN path, length(path) AS distance, nodes(path) AS pathNodes, relationships(path) AS pathEdges
Find Most Connected Movie	Finds movie with most connections	MATCH (m:Movie)<-[r]-(p:Person) RETURN m.title AS movie, count(r) AS connections ORDER BY connections DESC LIMIT 1

Find Debut Actors by Year	Finds actors who debuted in specific year	MATCH (a:Person)-[:ACTED_IN]->(m:Movie) WHERE m.released = {year} RETURN DISTINCT a.name AS actor, collect(m.title) AS movies
Find Actor Clusters	Finds clusters of actors based on collaborations	MATCH (a:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(b:Person) WHERE a.name < b.name WITH m.title AS movie, collect(DISTINCT a.name) + collect(DISTINCT b.name) AS actors RETURN movie, actors
Find Top Producers	Finds producers with most movies	MATCH (p:Person)-[:PRODUCED]->(m:Movie) RETURN p.name AS producer, count(m) AS movieCount ORDER BY movieCount DESC LIMIT 10

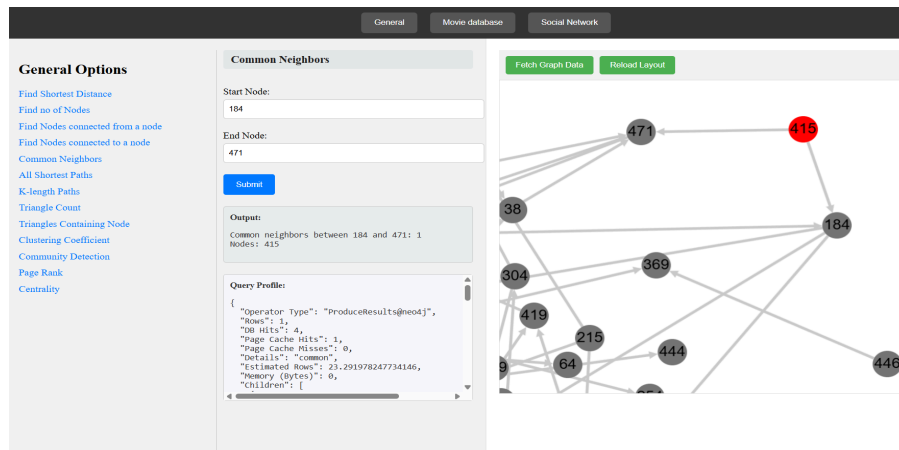
Results/Screenshots for some Queries



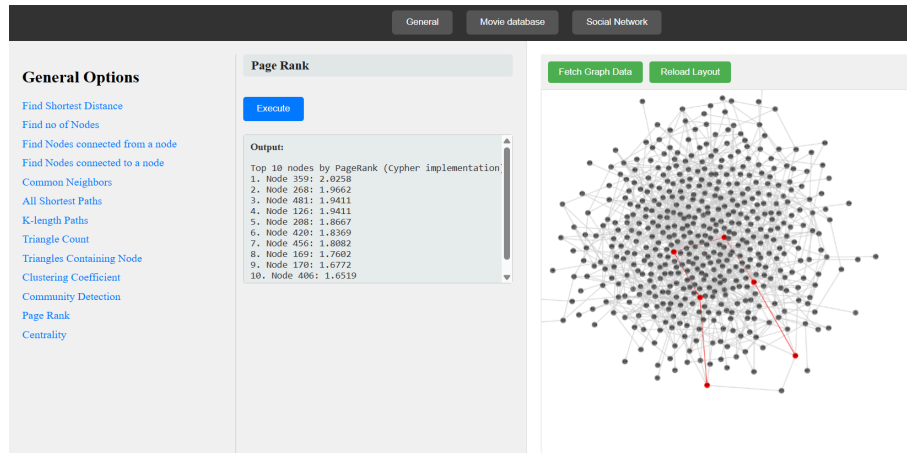
General graph database (nodes, edges)



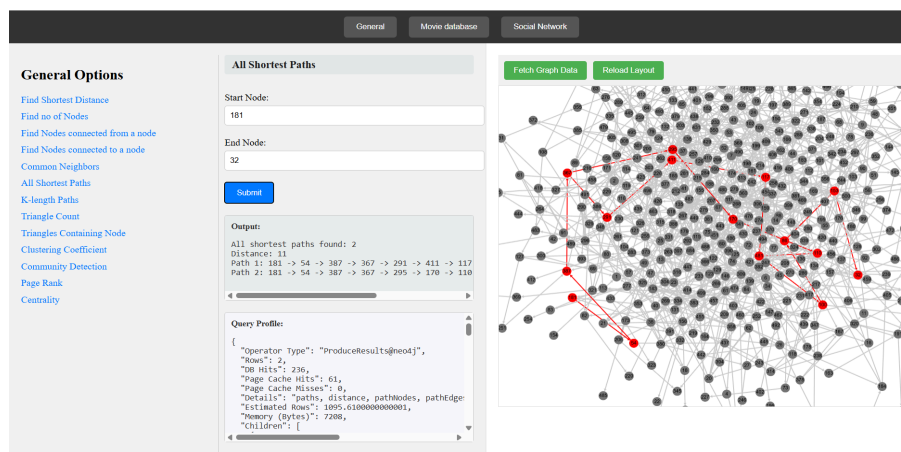
Shortest Distance b/w 1 and 2



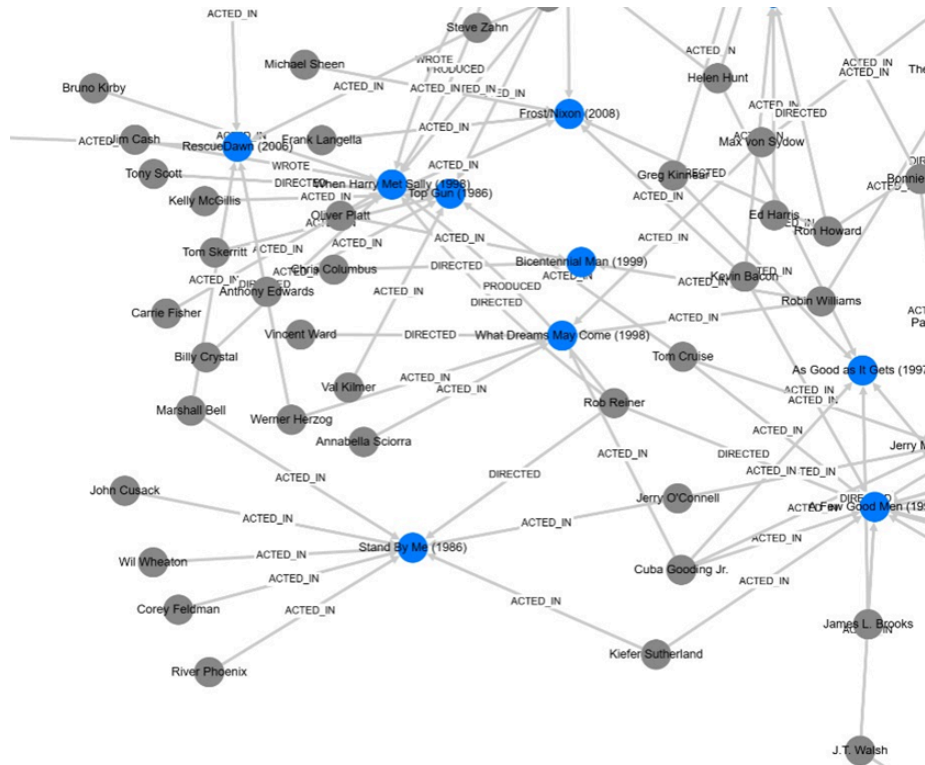
Common Neighbours for 184 and 471



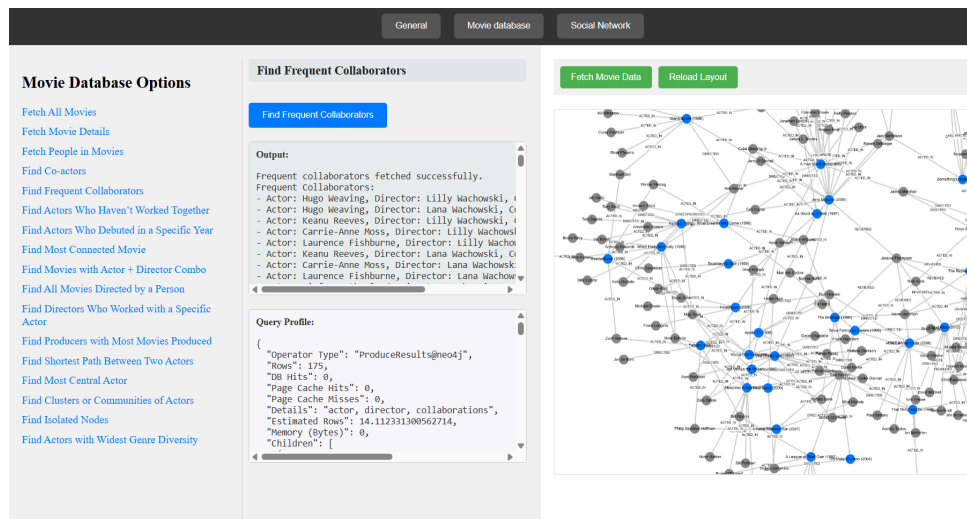
Page Rank (top 10 nodes with highest PageRank are displayed)



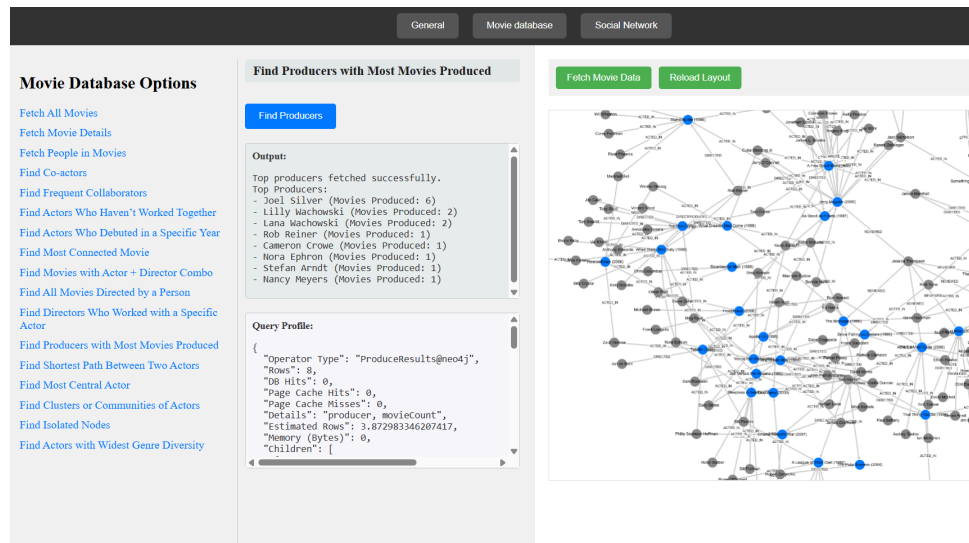
All Shortest Paths b/w 181 and 32



Movie DataBase



Frequent Collaborators



Producers with most movies produced

Future Scope for Graph DataBases

Natural Language Query Interface using LLMs

Instead of requiring users to learn Cypher, we can integrate a **large language model (LLM)** to interpret natural language queries and translate them into graph queries automatically. This would make the system more accessible to non-technical users and enhance usability.

Real-Time Data Integration

Future work can involve integrating live data sources such as movie APIs or social networks to keep the graph database updated in real-time, enabling dynamic queries and analytics.

Recommendation Systems

The underlying graph structure can support future development of recommendation systems (e.g., suggesting movies based on user preferences or finding similar directors), leveraging relationships in the graph.

Domain Expansion

Beyond entertainment, similar graph models can be applied in healthcare (linking patients, diseases, treatments), finance (transactions, fraud detection), and social media (user connections, influence spread), making graph databases crucial in modern data-driven applications.

Semantic Search & Knowledge Graphs

Building a semantic layer on top of the graph (like in knowledge graphs used by Google or LinkedIn) enables more intelligent search and inference, opening possibilities for powerful question-answering systems.

Hybrid Query Engines

Combining relational and graph queries in hybrid systems allows developers to benefit from the strengths of both worlds — structure from SQL and relationship exploration from graphs.

Integration with AI and Machine Learning Pipelines

Graph embeddings (like **Node2Vec**, **GraphSAGE**) can be used to feed ML models for node classification, link prediction, and clustering. This can enhance personalization, fraud detection, and recommendation tasks.

Conclusion

Through this project, we got hands-on experience with how graph databases work and why they're so useful for dealing with interconnected data. Using **Neo4j**, we built and explored different kinds of graphs — from large datasets like those in the Stanford SNAP repository to more real-life examples like a movie database where movies are linked to actors, directors, and reviewers.

We tried out interesting algorithms like **PageRank** and **community detection** to see how graph theory can help find patterns and useful insights. Along the way, we also tested how well Neo4j performs and handles big graphs, which gave us a better understanding of its strengths.

This project showed us how powerful graph databases are, especially for real-world scenarios where data relationships matter.

References

PageRank – <https://en.wikipedia.org/wiki/PageRank>

Neo4j GitHub – <https://github.com/neo4j/neo4j>

Clustering Coefficient – https://en.wikipedia.org/wiki/Clustering_coefficient

Louvain Method – https://en.wikipedia.org/wiki/Louvain_method

Stanford SNAP Datasets – <https://snap.stanford.edu/data/>